



Laboratoire Informatique d'Avignon
Stage 2026

Optimisation de Positionnement de points par Programmation Non Linéaire en Julia

Résolution du problème de géométrie de distances euclidiennes (EDGP)

Formulations testées

- Formulation 1 G1 : Q aléatoire
- Formulation 1 G2 : Q calculé à partir de X

Mai 2026

Table des matières

1	Introduction	2
1.1	Contexte physique	2
1.2	Problème mathématique	2
1.3	Environnement de développement	2
2	Formulation 1 – G1 : Q aléatoire	4
2.1	Présentation	4
2.2	Paramètres	4
2.3	Code	4
3	Formulation 1 – G2 : Q calculé à partir de X	6
3.1	Présentation	6
3.2	Paramètres	6
3.3	Code	6
3.4	Résultats numériques	8
3.5	Résultat graphique	10
4	Visualisations des résultats	11
4.1	Tableau de bord principal (Formulation 1)	11
4.2	les 10 premières simulations	12
4.3	Tableau récapitulatif des simulations	13
5	Analyse comparative	14
5.1	Récapitulatif des formulations	14
5.2	Bilan	14

1. Introduction

1.1 Contexte physique

Dans le domaine de l'informatique quantique avec des atomes de Rydberg, deux atomes en interaction produisent une énergie d'échange régie par la **loi de van der Waals** :

$$Q_{ij} = \frac{C_6}{\|\mathbf{x}_i - \mathbf{x}_j\|^6}$$

où C_6 est la constante de couplage (fixée à $C_6 = 1$) et $\mathbf{x}_i \in \mathbb{R}^d$ est la position de l'atome i .

1.2 Problème mathématique

Étant donnée une matrice de couplages $Q \in \mathbb{R}^{n \times n}$ (symétrique, diagonale nulle), le problème consiste à retrouver des positions $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ compatibles avec ces couplages. Ce problème est connu sous le nom de **problème de géométrie de distances euclidiennes** (*Euclidean Distance Geometry Problem*, EDGP).

Dans les formulations testées, Q_{ij} représente les **distances au carré** :

$$Q_{ij} = \|\mathbf{x}_i - \mathbf{x}_j\|^2 = \sum_{k=1}^d (x_{ik} - x_{jk})^2$$

et l'on cherche à minimiser un critère de résidu quadratique.

1.3 Environnement de développement

Langage	Julia
Bibliothèque	JuMP.jl
Solveur NLP	Ipopt (Interior Point OPTimizer)
Visualisation	Plots.jl
Configuration	$d = 2$ représente la dimension, $n \in \{3, \dots, 10\}$ est le nombre d'éléments de la matrice, sachant que la matrice est de taille $n \times n$

Les paramètres du modèle ont la signification suivante :

- L représente la plage des valeurs de X : si $L = 10$, les coordonnées de X sont comprises entre 0 et 10.
- d représente la dimension étudiée. Ici le problème est résolu en dimension 2.
- n représente le nombre de points.
- Le **nombre d'instances** représente le nombre de fois où Q est tirée avec les mêmes n et d . Par exemple, si $n = 3$, $d = 2$ et le nombre de tirages est 10, on obtient 10 matrices différentes de taille 3×3 , et le X généré est de dimension $n \times 2$.

Fichier	Étiquette	Description
formulation_1_g1.jl	F1-G1	Distances euclidiennes, Q aléatoire
formulation_1_g2.jl	F1-G2	Distances euclidiennes, vérité terrain

Table 1 – Fichiers Julia couverts par ce rapport.

2. Formulation 1 – G1 : Q aléatoire

2.1 Présentation

Cette première formulation constitue le point de départ. La matrice Q est générée **aléatoirement** : il n'existe pas de positions réelles sous-jacentes. On cherche des positions $\mathbf{x}^* \in \mathbb{R}^{n \times 2}$ qui satisfont les distances imposées par Q .

Formulation mathématique

$$\min_x \sum_{i=1}^n \sum_{j=i+1}^n (\|x_i - x_j\|^2 - q_{ij}^2)^2$$

avec $q_{ij} \sim \mathcal{U}([0, 10])$ aléatoire, $q_{ij} = q_{ji}$, $q_{ii} = 0$.

2.2 Paramètres

Paramètre	Valeur
Plage de n	3 à 10
Dimension d	2
Tirages par instance	10
Borne supérieure L	10
Solveur	Ipopt

2.3 Code

```

1 using JuMP, Ipopt, Plots, LinearAlgebra
2
3 limit_points = 10; limit_dim = 2
4 number_generation_per_instance = 10; L = 10
5
6 for n in 3:limit_points
7     for dim in 2:limit_dim
8         for _ in 1:number_generation_per_instance
9             Q = zeros(n, n)
10            for i in 1:n
11                for j in i+1:n
12                    gen_Q = rand(0:0.01:L)
13                    Q[i,j] = gen_Q
14                    Q[j,i] = gen_Q
15                end
16            end
17
18            model = Model(Ipopt.Optimizer)
19            set_optimizer_attribute(model, "print_level", 0)
20            @variable(model, x_opt[1:n, 1:dim] >= 0)
21            @objective(model, Min, sum(
22                (sum((x_opt[i,k] - x_opt[j,k])^2 for k in 1:dim)
23                  - Q[i,j]^2)^2

```

```
24         for i in 1:n, j in i+1:n))
25
26         optimize!(model)
27         println("objective_value = ", objective_value(model))
28
29         scatter!(value.(x_opt)[: ,1], value.(x_opt)[: ,2], label="
30                 x_opt")
31         savefig("comparaison_mds1.png")
32     end
33 end
```

Listing 1 – formulation_1_g1.jl

3. Formulation 1 – G2 : Q calculé à partir de X

3.1 Présentation

La formulation G2 améliore G1 en s'appuyant sur une **vérité terrain** : des positions réelles $\mathbf{x}^{\text{reel}}$ sont d'abord générées, la matrice Q est calculée à partir des distances au carré exactes, puis le solveur doit retrouver ces positions.

Formulation mathématique

Étape 1 – construction de Q :

$$q_{ij} = \|\mathbf{x}_i^{\text{reel}} - \mathbf{x}_j^{\text{reel}}\|^2$$

Étape 2 – optimisation :

$$\min_x \sum_{i=1}^n \sum_{j=i+1}^n (\|x_i - x_j\|^2 - q_{ij})^2$$

3.2 Paramètres

Paramètre	Valeur
Graine aléatoire	Random.seed!(2024)
Plage de n	3 à 10
Dimension d	2
Tirages par instance	10
Borne L	10
Sorties	results.csv, comparaison_mds.png

Ici, Random.seed!(2024) consiste à rendre les résultats reproductibles sur la même machine : c'est-à-dire qu'une fois les X générés aléatoirement, si le programme est recompilé, les valeurs tirées restent les mêmes.

3.3 Code

```

1 using Random, JuMP, Ipopt, Plots, LinearAlgebra
2 Random.seed!(2024)
3
4 L = 10; limit_points = 10; limit_dim = 2
5 number_generation_per_instance = 10
6
7 for n in 3:limit_points
8     for dim in 2:limit_dim
9         for _ in 1:number_generation_per_instance
10             x = L * rand(n, dim) # positions reelles (verite
11                                     terrain)
12             Q = zeros(n, n)
13             for i in 1:n

```

```

13     for j in i+1:n
14         norm_value = sum((x[i,:] - x[j,:]).^2) # distance au
           carre
15         Q[i,j] = norm_value
16         Q[j,i] = norm_value
17     end
18 end
19
20 model = Model(Ipopt.Optimizer)
21 @variable(model, x_opt[1:n, 1:dim] >= 0)
22 @objective(model, Min, sum(
23     (sum((x_opt[i,k] - x_opt[j,k])^2 for k in 1:dim)
24     - Q[i,j])^2
25     for i in 1:n, j in i+1:n))
26
27 optimize!(model)
28
29 p = scatter(x[:,1], x[:,2], label="x_reel")
30 scatter!(value.(x_opt)[:,1], value.(x_opt)[:,2], label="
      x_opt")
31 savefig("comparaison_mds.png")
32
33 open("results.csv", "a") do file
34     println(file, "x_reel : ", x)
35     println(file, "Q : ", Q)
36     println(file, "Valeur de l'objectif : ", objective_value(
      model))
37     println(file, "x_opt = ", value.(x_opt))
38 end
39 end
40 end
41 end

```

Listing 2 – formulation_1_g2.jl

3.4 Résultats numériques

Instance	n	Valeur de l'objectif	Qualité
1	3	$1,72 \times 10^{-19}$	Excellente
2	3	$1,01 \times 10^{-13}$	Très bonne
3	3	$6,58 \times 10^{-18}$	Excellente
4	3	$1,66 \times 10^{-16}$	Excellente
5	3	$2,37 \times 10^{-22}$	Excellente
6	4	$1,22 \times 10^{-19}$	Excellente
7	5	$1,76 \times 10^{-19}$	Excellente
8	6	$1,79 \times 10^{-19}$	Excellente
9	7	$2,50 \times 10^{-19}$	Excellente
10	300	$5,17 \times 10^{-22}$	Excellente

Table 2 – Valeurs de la fonction objectif pour F1-G2 (extrait de `results.csv`). Une valeur $< 10^{-10}$ traduit une convergence quasi-parfaite.

Ce tableau montre une convergence avec Ipopt même pour des instances avec $n = 300$, ce qui signifie que même pour une matrice de taille 300×300 on arrive à trouver une solution optimale.

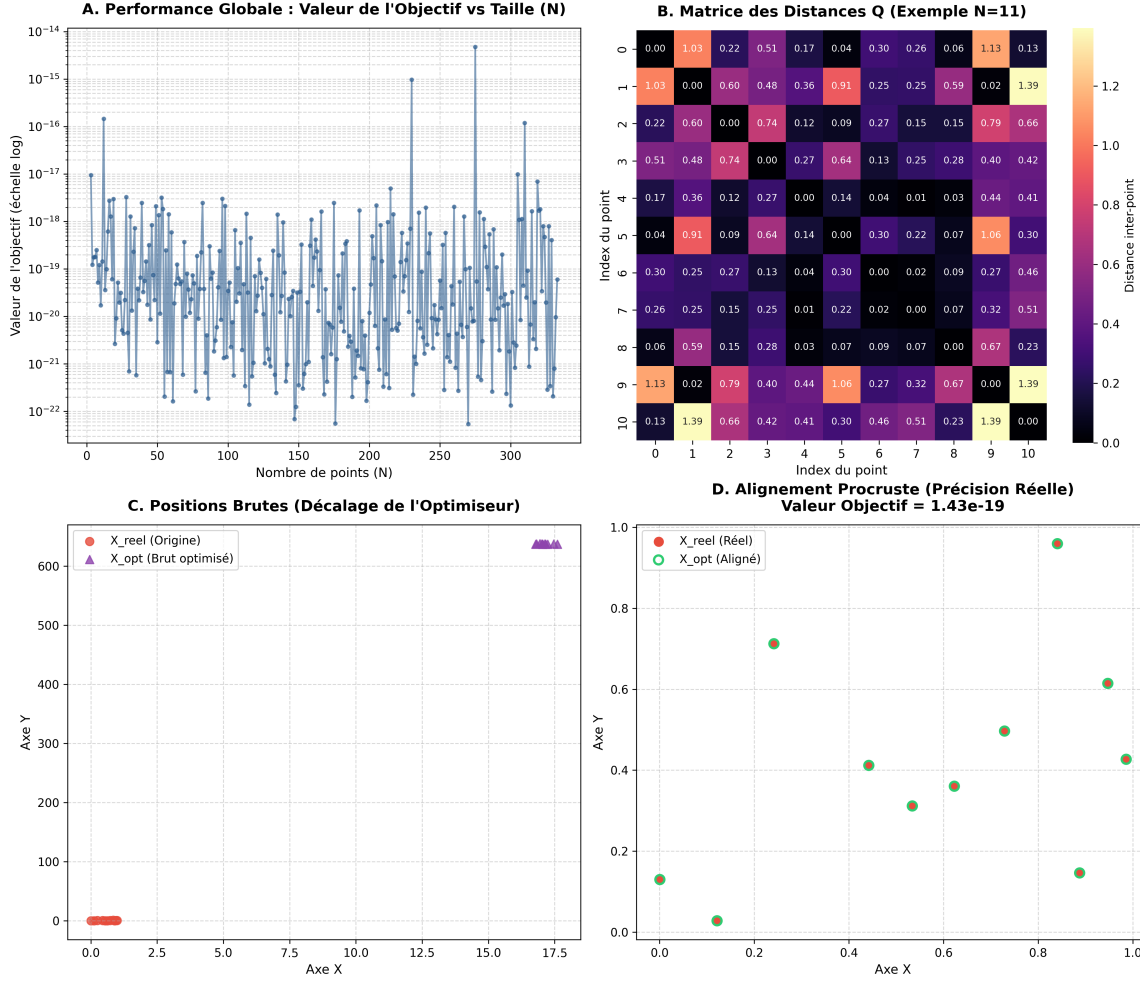


Figure 1 – Tableau de bord pour n allant jusqu'à 300. Le graphe A montre la valeur de l'objectif en fonction du nombre de points N : la convergence reste excellente ($\sim 10^{-19}$ à 10^{-22}) sur l'ensemble des instances. Le graphe B présente la matrice Q pour $N = 11$. Le graphe C illustre le décalage brut entre $\mathbf{x}^{\text{reel}}$ et $\mathbf{x}^{\text{opt}}$ dû à l'invariance par translation. Le graphe D montre l'alignement Procruste : après correction par SVD, les deux nuages de points se superposent parfaitement (valeur objectif = $1,43 \times 10^{-19}$).

Phénomène de translation : les positions retrouvées $\mathbf{x}^*$ sont translatées par rapport à $\mathbf{x}^{\text{reel}}$ (par exemple $\mathbf{x}^* \approx (1149, 1156)$ au lieu de valeurs dans $[0, 10]^2$). Cela s'explique par l'**invariance par translation** du problème : les distances au carré sont insensibles au décalage du repère. Il s'agit d'un comportement attendu et non d'une erreur.

La valeur de l'objectif garantit la convergence. De plus, si l'on regarde de plus près, on remarque que la distribution des points de la solution réelle et celle de la solution $\mathbf{x}^{\text{opt}}$ trouvée sont quasiment identiques, mais avec une translation différente.

3.5 Résultat graphique

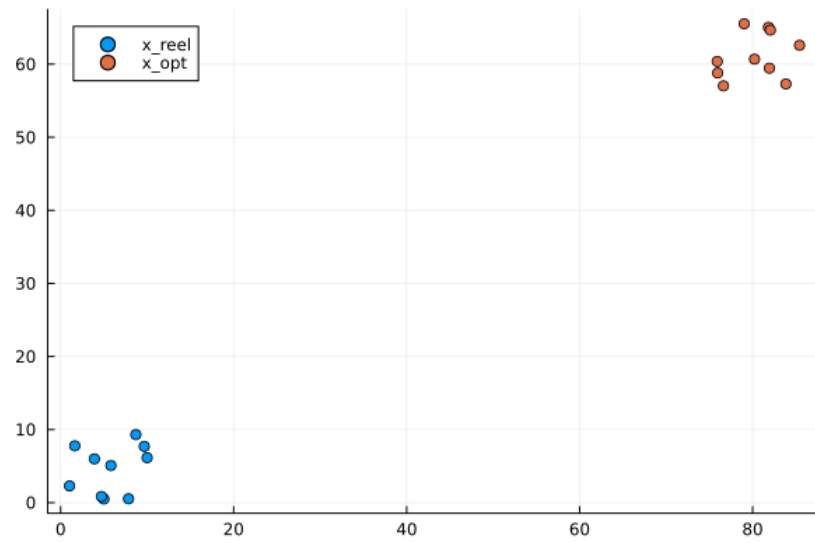


Figure 2 – Comparaison entre les positions réelles (x_{reel}) et les positions optimisées (x_{opt}) pour F1-G2. La **configuration géométrique est parfaitement préservée** ; seule la translation absolue diffère.

4. Visualisations des résultats

4.1 Tableau de bord principal (Formulation 1)

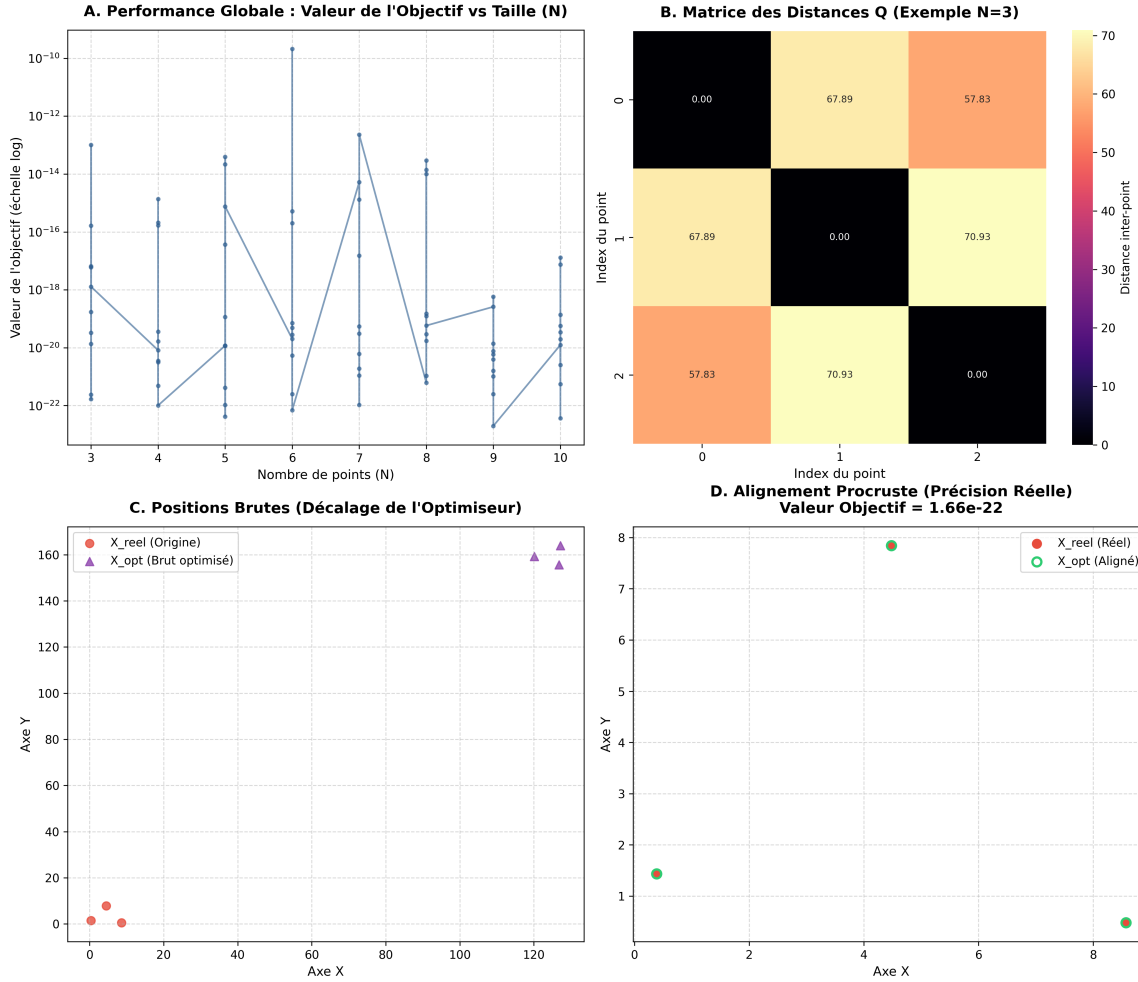


Figure 3 – Tableau de bord des résultats d'optimisation (Formulation 1). Sur l'axe des abscisses figure le nombre de points N , et le nombre d'instances est de 10 pour chaque valeur de N et sur l'axe des ordonnées la valeur de l'objectif obtenue. L'alignement $\mathbf{x}^{\text{opt}}$ sur $\mathbf{x}^{\text{reel}}$ est réalisé par décomposition en valeurs singulières (SVD), ce qui permet de retrouver les valeurs de X en supprimant la translation et en centrant les deux configurations.

4.2 les 10 premières simulations

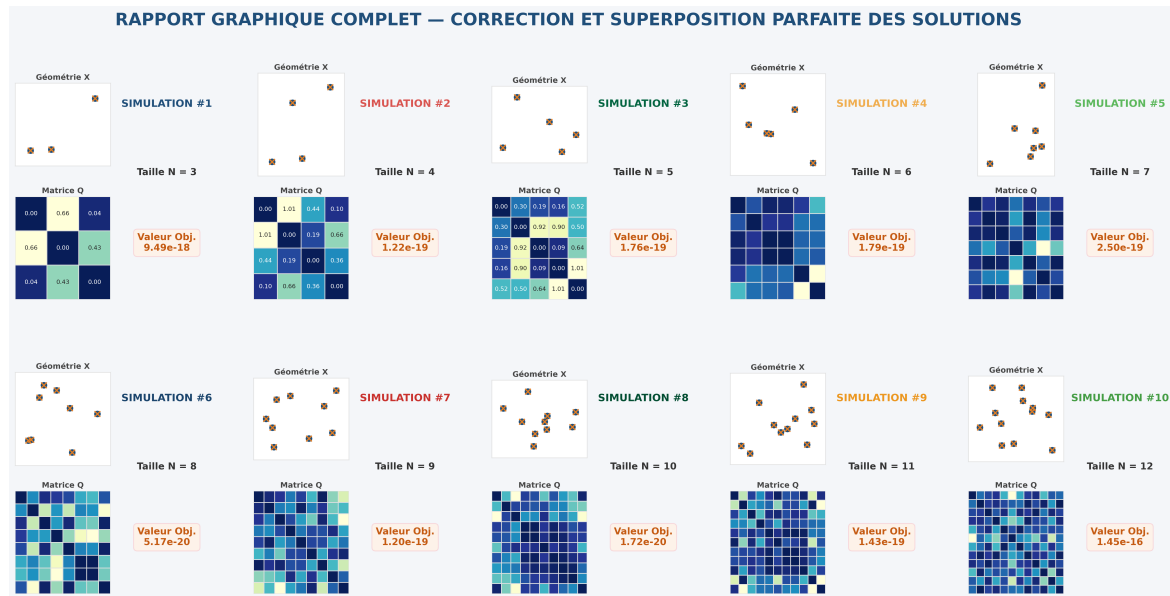


Figure 4 – Les 10 premières simulations avec les valeurs d’objectif obtenues, n variant de 1 à 10. Ces configurations correspondent aux instances pour lesquelles Ipopt a convergé le plus précisément.

4.3 Tableau récapitulatif des simulations

TABLEAU RÉCAPITULATIF DES 10 PREMIÈRES SIMULATIONS

ID (N)	Valeur Objectif	Matrice Q	X Optimisé
#1 (N=3)	-3.1974e-14	[9.00 10.00 10.00] [10.00 0.00 9.00] [10.00 9.00 0.00]	[100.107 100.121] [105.168 107.214] [107.289 105.092]
#2 (N=4)	9.6784e+00	[0.00 1.00 8.00 3.00] [1.00 0.00 0.00 4.00] [0.00 0.00 0.00 1.00] [3.00 4.00 1.00 0.00]	[40.224 292.550] [39.688 291.113] [40.362 290.950] [41.365 290.973]
#3 (N=5)	4.7720e+01	[0.00 5.00 10.00 0.00 10.00] [5.00 0.00 5.00 0.00 7.00] [10.00 5.00 0.00 0.00 9.00] [0.00 0.00 0.00 0.00 6.00] [10.00 7.00 9.00 6.00 0.00]	[16.510 21.048] [15.410 19.979] [14.136 22.443] [15.071 21.890] [13.353 19.819]
#4 (N=6)	6.3164e+01	[0.00 1.00 0.00 7.00 8.00 2.00] [1.00 0.00 0.00 0.00 9.00 8.00] [0.00 0.00 0.00 0.00 10.00 9.00] [7.00 0.00 0.00 0.00 2.00 2.00] [8.00 9.00 10.00 2.00 0.00 3.00] [2.00 8.00 9.00 2.00 3.00 0.00]	[795.551 631.152] [793.736 630.440] [794.058 631.512] [794.545 629.686] [795.515 628.294] [796.366 629.362]
#5 (N=7)	4.4409e+01	[0.00 4.00 4.00 4.00 1.00 1.00 9.00] [4.00 0.00 8.00 0.00 4.00 7.00 0.00] [4.00 0.00 0.00 5.00 0.00 9.00 4.00] [4.00 6.00 5.00 0.00 2.00 4.00 7.00] [1.00 4.00 0.00 7.00 0.00 1.00 1.00] [1.00 7.00 9.00 4.00 1.00 0.00 7.00] [9.00 0.00 4.00 7.00 1.00 7.00 0.00]	[974.818 450.502] [976.966 451.556] [974.724 452.984] [974.349 451.265] [975.878 450.527] [975.229 449.823] [976.831 452.068]
#6 (N=8)	1.0199e+02	[0.00 1.00 9.00 3.00 5.00 2.00 9.00 0.00] [1.00 0.00 0.00 9.00 3.00 8.00 1.00 0.00] [9.00 0.00 0.00 4.00 2.00 0.00 0.00 0.00] [3.00 9.00 4.00 0.00 4.00 1.00 4.00 5.00] [5.00 3.00 2.00 0.00 0.00 9.00 7.00 3.00] [2.00 8.00 0.00 1.00 9.00 0.00 6.00 8.00] [9.00 1.00 0.00 4.00 7.00 6.00 0.00 0.00] [0.00 8.00 0.00 5.00 3.00 0.00 9.00 0.00]	[0.131 1.296] [1.196 3.072] [2.910 1.258] [1.847 0.323] [1.110 2.588] [1.543 0.160] [2.968 2.096] [0.196 0.696]
#7 (N=9)	1.8072e+02	[0.00 7.00 10.00 0.00 6.00 4.00 10.00 0.00 6.00] [7.00 0.00 5.00 5.00 10.00 3.00 8.00 6.00 7.00] [10.00 5.00 0.00 5.00 6.00 5.00 4.00 3.00 9.00] [0.00 5.00 5.00 0.00 5.00 10.00 7.00 3.00 4.00] [6.00 10.00 6.00 5.00 0.00 6.00 6.00 10.00 6.00] [4.00 3.00 5.00 10.00 5.00 0.00 3.00 2.00 1.00] [10.00 0.00 4.00 7.00 6.00 3.00 0.00 7.00 1.00] [0.00 6.00 3.00 3.00 10.00 2.00 7.00 0.00 5.00] [6.00 7.00 9.00 4.00 6.00 1.00 1.00 5.00 0.00]	[212.105 121.642] [210.272 122.982] [210.687 124.264] [211.708 121.080] [213.490 122.827] [211.906 124.247] [212.656 124.756] [210.531 123.612] [212.927 123.959]
#8 (N=10)	1.9269e+02	[0.00 3.00 4.00 9.00 5.00 3.00 4.00 0.00 3.00 3.00] [3.00 0.00 0.00 7.00 3.00 6.00 10.00 3.00 8.00 0.00] [4.00 0.00 0.00 10.00 4.00 7.00 1.00 10.00 6.00 3.00] [9.00 7.00 10.00 0.00 8.00 3.00 10.00 4.00 3.00 4.00] [5.00 3.00 4.00 0.00 0.00 7.00 1.00 1.00 4.00 5.00] [3.00 6.00 7.00 3.00 7.00 0.00 5.00 9.00 3.00 9.00] [4.00 10.00 1.00 10.00 1.00 5.00 0.00 6.00 2.00 10.00] [0.00 3.00 10.00 4.00 1.00 9.00 6.00 0.00 5.00 6.00] [3.00 8.00 6.00 3.00 4.00 3.00 2.00 5.00 0.00 8.00] [3.00 0.00 3.00 4.00 5.00 5.00 10.00 6.00 8.00 0.00]	[82.872 81.902] [83.823 82.173] [82.448 83.107] [82.657 79.783] [82.372 82.655] [81.277 80.652] [81.105 82.557] [83.551 80.944] [81.319 81.055] [84.109 81.006]

Figure 5 – Tableau complet de toutes les simulations exécutées : chaque ligne correspond à une instance (n , tirage), avec la valeur de l’objectif atteinte.

5. Analyse comparative

5.1 Récapitulatif des formulations

Version	Objectif	Matrice Q	Convergence
F1-G1	$\sum_{i < j} (\ \Delta \mathbf{x}_{ij}\ ^2 - Q_{ij}^2)^2$	Aléatoire continu	Bonne (converge parfois mais pas toujours, car les valeurs générées ne respectent pas toujours les contraintes géométriques euclidiennes)
F1-G2	$\sum_{i < j} (\ \Delta \mathbf{x}_{ij}\ ^2 - Q_{ij})^2$	Distances au carré réelles	Excellente ($< 10^{-10}$)

Table 3 – Synthèse des formulations testées.

5.2 Bilan

Ce travail a permis de construire et tester deux formulations du problème de positionnement en Julia avec JuMP/Ipopt.

Les résultats montrent que :

- **F1-G2** (vérité terrain) atteint des valeurs d'objectif de l'ordre de 10^{-19} à 10^{-22} , ce qui confirme la quasi-parfaite reconstruction des positions originales à une isométrie (translation, rotation, réflexion) près.
- **F1-G1** (Q aléatoire) converge vers une solution locale satisfaisant les distances imposées, mais sans garantie de réalisabilité géométrique puisque Q n'est pas nécessairement issue d'un plongement euclidien réel.
- Le phénomène de **translation** observé en F1-G2 est intrinsèque au problème et non une erreur : les distances au carré sont invariantes par translation du repère. Un alignement de Procruste permet de superposer visuellement les configurations.
- Ipopt se comporte de manière robuste sur cette classe de problèmes NLP non convexes, même avec $n = 300$, avec une convergence systématiquement excellente en F1-G2.

La suite consiste à essayer d'augmenter n jusqu'à 1000 pour vérifier si la convergence se maintient, et à passer en dimension $d = 3$.